

UART protocol

The protocol for the communication between ESP32 and RP2040 (on board MCU).

Protocol version: 1.0

Protocol date: 23.03.2026

Features

The protocol should cover following points:

- Controller support:
 - Inputs from a wireless controller (connected to ESP32), like stick positions, button events
 - Actions/Feedback to the controller, like rumble, led lights, ...
- ESP NOW networking:
 - Receive messages
 - Send messages (broad- and unicast)
- Data integrity:
 - Use of checksums to check for transmission errors
 - Use of timeouts for reset procedures

UART Implementation

Hardware side

Hardwarewise the following pins are used:

RP2040		ESP32
UART0 TX @ GP28	-----	RX @ IO16
UART0 RX @ GP29	-----	TX @ IO17

Software side

Following code can be used to initialize the Serial objects with the Arduino framework:

Micromouse (RP2040)

```
// Global variable
arduino::UART Serial2(*TX*/28,*RX*/29, /*RTS*/NC, /*CTS*/NC);

void setup(){
    Serial2.begin(9600, SERIAL_8N1);
}
```

ESP32

```
void setup(){
    Serial1.begin(9600, SERIAL_8N1, /*RX*/16, /*TX*/17);
}
```

A connection speed of **9600 BAUD** is used due to stability reasons.

The Protocol

The ESP and the RP can both start to send data. There is no Master/Slave setup.

For the communication following format is used:

OpCode	Length (Payload)	Value/Values (0-250) (=Payload)	Checksum
see table	8 Bit	0-255 x 8 Bit	8 Bit sum

After receiving one packet the checksum should be calculated and compared with the transmitted one. In case of difference the packet can be requested again with a NAK Bit.

After receiving the receiver should reply to the transmitter ACK (ACKnowledgement) or NAK (No Acknowledgement). ACK and NAK should be send like every other message with the defined frame ([OpCode][Length][Value (optional)][Checksum]).

OpCode

Code	Name	Values [bytes]	Description
11	Stick L	2 [X, Y]	Left Stick
12	Stick R	2 [X, Y]	Right Stick
13	L2 Trigger	1	Left 2 Trigger (analog)
14	R2 Trigger	1	Right 2 Trigger (analog)
15	SW_Set	2	Switches set, see below
16	Gyroscope	3 [X, Y, Z]	
31	RGB LED	3 [RGB]	
32	Player LED	1	Player indicator LED
33	Rumble	1	0
101	Send broadcast	0-250 (message)	Send message to all ESPs
102	Send unicast	1 (target) 0-250 (msg)	Send to one ESP.
110	Msg received	0-255 (message)	Message from other
200	ACK		Acknowledgement
201	NAK	1 error code	No Acknowledgement
211	Enable Subscription	2 [ID, threshhold]	Enable change notification of element (ID = OpCode)
212	Disable Subscription	ID	Disable change notification
213	Reset Subscriptions		Disable all change notifications
214	Request current value	ID	Request message (ID = OpCode)
215	Ping		Sends ping, expects answer (216)
216	Ping answer		Answer to Ping (215)

SW_Set

Bit meanings (Pressed when 1) first Byte:

1. L1 Trigger
2. R1 Trigger
3. Share button
4. Options button
5. PS button
6. Mute button
7. Left stick button
8. Right stick button

Bit meanings (Pressed when 1) second Byte:

1. Cross up
2. Cross right
3. Cross down
4. Cross left
5. Triangle
6. Circle
7. X
8. Square

Error codes

MSB set: request last message again

Code	Name	Description	Example
1	Checksum	Checksum does not match	
2	Invalid OpCode	Opcode invalid	OpCode 5
3	Invalid value	Value range doesn't match	Player LED 200
4	Invalid length	Unexpected value length	

Length

This byte defines the number of payload bytes. For protocol version 1.0 it's only possible to send up to 255 payload bytes (should be enough, ESP NOW doesn't support more than 250 bytes anyways...).

Because of the extra header (OpCode + Length) and checksum every message is 3 bytes longer than the length byte.

Values / Payload

Here is the opcode dependent data located.

Checksum

The checksum is calculated as the sum of all bytes (OpCode, length, Payload) except the checksum itself. It should be interpreted as an unsigned 8-bit integer (uint8_t).

Further concepts

Ideas behind the protocol

Subscription

The protocol supports subscription to changes. This is especially handy for controller inputs. The RP2040 can subscribe to parameters and get messages when something changes. The threshold specifies the amount of the change for analog values and is ignored for digital ones.

Timeouts and safety

There are following timeouts and procedures:

- **Length timeout** (receiver side): if the message is not finished after **100 ms** the NAK (invalid length) with resend request should be sent. A maximum of **5 retries** is allowed before going into error state.
- **Answer timeout** (transmitter side): if the communication partner doesn't answer at all in **100 ms** goto error state and retry connection every second.
- **Checksum & ACK system**: If something goes wrong during the communication the checksum should catch some of the errors.